

ML Loop Code-Along - Step 07

Deliberately overfit something

Incremental prerequisite prep for Imperial ML/AI coursework

Prepared for Anthony Watts - June 2026

Rule: nothing appears before it is explained.

7. Deliberately overfit something

Overfitting becomes easier to recognise when you make it happen on purpose. A very flexible decision tree can carve the training data into tiny rules and look brilliant on examples it has already seen.

Step 7.1: Import a flexible regression model

Why this cell exists

DecisionTreeRegressor can model non-linear patterns. If unconstrained, it can also memorise training data too closely.

```
from sklearn.tree import DecisionTreeRegressor
```

What to look for

This is a different model class, but it still follows the same fit/predict pattern.

Step 7.2: Fit an unconstrained tree

Why this cell exists

No `max_depth` means the tree is allowed to grow until it can fit the training data very closely.

```
wild_tree = DecisionTreeRegressor(random_state=42)
wild_tree.fit(X_reg_train, y_reg_train)

wild_train_predictions = wild_tree.predict(X_reg_train)
wild_test_predictions = wild_tree.predict(X_reg_test)
```

What to look for

This cell trains a model that is intentionally suspicious. The next cell measures the damage.

Step 7.3: Compare its train and test scores

Why this cell exists

A near-perfect training score does not prove a good model. It may simply prove the model had enough flexibility to remember the training set.

```
print("Wild tree train R2:", round(
    r2_score(y_reg_train, wild_train_predictions), 3
))

print("Wild tree test R2:", round(
    r2_score(y_reg_test, wild_test_predictions), 3
))
```

What to look for

This is the numerical version of: it looked clever on familiar examples and less clever on new ones.

Step 7.4: Try a constrained tree

Why this cell exists

`max_depth` limits how detailed the tree is allowed to become. This can reduce memorisation, although it does not guarantee the best model.

```
small_tree = DecisionTreeRegressor(  
    max_depth=3,  
    random_state=42,  
)  
  
small_tree.fit(X_reg_train, y_reg_train)  
  
small_train_predictions = small_tree.predict(X_reg_train)  
small_test_predictions = small_tree.predict(X_reg_test)  
  
print("Small tree train R2:", round(  
    r2_score(y_reg_train, small_train_predictions), 3  
)  
)  
  
print("Small tree test R2:", round(  
    r2_score(y_reg_test, small_test_predictions), 3  
)  
)
```

What to look for

The point is not that `max_depth=3` is perfect. The point is that model complexity changes the train/test relationship.

Checkpoint

You should now be able to explain: high training score alone is not the goal. The goal is useful performance on unseen examples.